COMP102 Lecture 11: Searching Hashing

Linear/binary search, hash functions, collisions, chaining, open addressing

Jalal Maaouni
June 17, 2026

Plan

1. Searching as an engineering problem
2. Linear search
3. Binary search
4. Dynamic ordered search
5. Hashing
6. Hash functions
7. Hash tables
8. Collision resolution: chaining
9. Collision resolution: open addressing
10. Conclusion

Searching as an engineering problem

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: “Where is the object with this exact identifier?”

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: "Where is the object with this exact identifier?"

- mechanical engineering: parts, machines, maintenance logs;

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: "Where is the object with this exact identifier?"

- mechanical engineering: parts, machines, maintenance logs;
- electrical engineering: sensors, circuits, devices, signals;

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: "Where is the object with this exact identifier?"

- mechanical engineering: parts, machines, maintenance logs;
- electrical engineering: sensors, circuits, devices, signals;
- chemical and materials engineering: batches, compounds, catalysts, samples;

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: "Where is the object with this exact identifier?"

- mechanical engineering: parts, machines, maintenance logs;
- electrical engineering: sensors, circuits, devices, signals;
- chemical and materials engineering: batches, compounds, catalysts, samples;
- environmental engineering: water stations, soil samples, climate alerts;

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: "Where is the object with this exact identifier?"

- mechanical engineering: parts, machines, maintenance logs;
- electrical engineering: sensors, circuits, devices, signals;
- chemical and materials engineering: batches, compounds, catalysts, samples;
- environmental engineering: water stations, soil samples, climate alerts;
- applied mathematics and business: clients, transactions, risks, indicators;

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: "Where is the object with this exact identifier?"

- mechanical engineering: parts, machines, maintenance logs;
- electrical engineering: sensors, circuits, devices, signals;
- chemical and materials engineering: batches, compounds, catalysts, samples;
- environmental engineering: water stations, soil samples, climate alerts;
- applied mathematics and business: clients, transactions, risks, indicators;
- industrial platforms: production orders, inventory items, quality reports.

Searching is everywhere in engineering

Core operation

Many engineering systems repeatedly ask: "Where is the object with this exact identifier?"

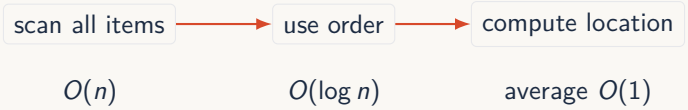
- mechanical engineering: parts, machines, maintenance logs;
- electrical engineering: sensors, circuits, devices, signals;
- chemical and materials engineering: batches, compounds, catalysts, samples;
- environmental engineering: water stations, soil samples, climate alerts;
- applied mathematics and business: clients, transactions, risks, indicators;
- industrial platforms: production orders, inventory items, quality reports.

Searching is not a secondary detail. In engineering systems, finding the right object quickly often determines monitoring, control, diagnosis, and decision speed.

Today: from scanning to direct access

Target

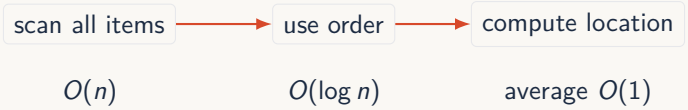
We want to move from checking elements one by one to computing where a key should be stored.



Today: from scanning to direct access

Target

We want to move from checking elements one by one to computing where a key should be stored.

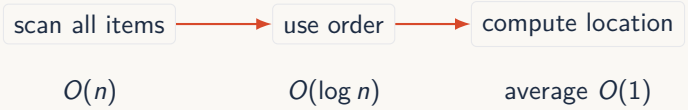


- linear search: no assumptions;

Today: from scanning to direct access

Target

We want to move from checking elements one by one to computing where a key should be stored.

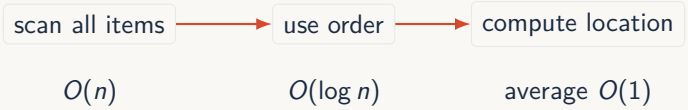


- linear search: no assumptions;
- binary search: sorted data;

Today: from scanning to direct access

Target

We want to move from checking elements one by one to computing where a key should be stored.

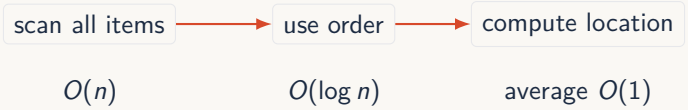


- linear search: no assumptions;
- binary search: sorted data;
- balanced trees: ordered dynamic data;

Today: from scanning to direct access

Target

We want to move from checking elements one by one to computing where a key should be stored.



- linear search: no assumptions;
- binary search: sorted data;
- balanced trees: ordered dynamic data;
- hashing: exact lookup by key.

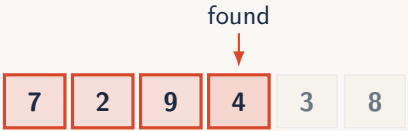
Linear search

Linear search: baseline strategy

Idea

Check elements one by one until the target is found or the list ends.

```
1 def contains(items, target):
2     for item in items:
3         if item == target:
4             return True
5     return False
```

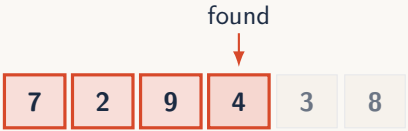


Linear search: baseline strategy

Idea

Check elements one by one until the target is found or the list ends.

```
1 def contains(items, target):
2     for item in items:
3         if item == target:
4             return True
5     return False
```



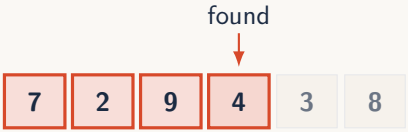
- best case: $O(1)$;

Linear search: baseline strategy

Idea

Check elements one by one until the target is found or the list ends.

```
1 def contains(items, target):
2     for item in items:
3         if item == target:
4             return True
5     return False
```



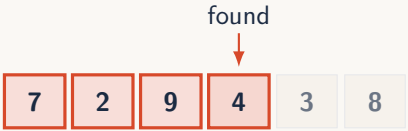
- best case: $O(1)$;
- worst case: $O(n)$;

Linear search: baseline strategy

Idea

Check elements one by one until the target is found or the list ends.

```
1 def contains(items, target):
2     for item in items:
3         if item == target:
4             return True
5     return False
```



- best case: $O(1)$;
- worst case: $O(n)$;
- average case: $O(n)$.

Linear search: average-case proof

Assumption

The target is equally likely to be at any position $1, 2, \ldots, n$.

Linear search: average-case proof

Assumption

The target is equally likely to be at any position $1, 2, \ldots, n$.

Average number of checks

$$\frac{1+2+3+\cdots+n}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

Linear search: average-case proof

Assumption

The target is equally likely to be at any position $1, 2, \ldots, n$.

Average number of checks

$$\frac{1+2+3+\cdots+n}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

The average number of checks grows linearly with n. Therefore average-case linear search is $\Theta(n)$.

Linear search: average-case proof

Assumption

The target is equally likely to be at any position $1, 2, \ldots, n$.

Average number of checks

$$\frac{1+2+3+\cdots+n}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

The average number of checks grows linearly with n. Therefore average-case linear search is $\Theta(n)$.

If the target may be absent

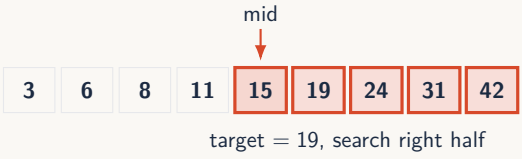
Absence requires scanning the whole list, so the conclusion remains linear.

Binary search

Binary search: fast search when order exists

Definition 3.1 Binary search

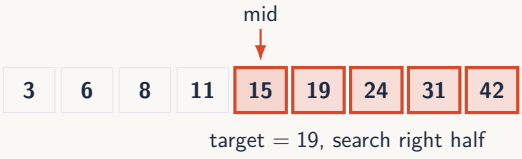
Binary search compares with the middle element and discards half of the remaining search space.



Binary search: fast search when order exists

Definition 3.2 Binary search

Binary search compares with the middle element and discards half of the remaining search space.

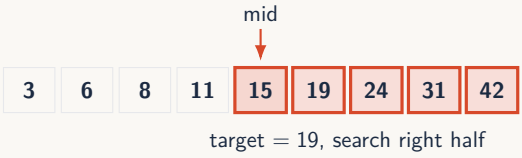


- requires sorted data;

Binary search: fast search when order exists

Definition 3.3 Binary search

Binary search compares with the middle element and discards half of the remaining search space.

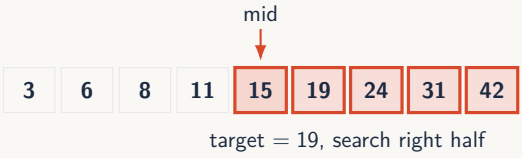


- requires sorted data;
- requires sorting by the search key;

Binary search: fast search when order exists

Definition 3.4 Binary search

Binary search compares with the middle element and discards half of the remaining search space.

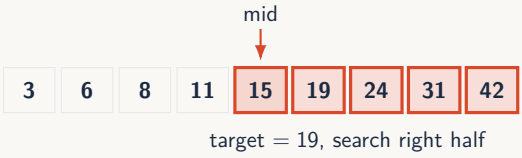


- requires sorted data;
- requires sorting by the search key;
- complexity: $O(\log n)$.

Binary search: fast search when order exists

Definition 3.5 Binary search

Binary search compares with the middle element and discards half of the remaining search space.



- requires sorted data;
- requires sorting by the search key;
- complexity: $O(\log n)$.

Binary search is powerful because every comparison gives a direction: left or right.

Coding: binary search

```
1  def binary_search(items, target):
2      left = 0
3      right = len(items) - 1
4
5      while left <= right:
6          mid = (left + right) // 2
7
8          if items[mid] == target:
9              return True
10         elif items[mid] < target:
11             left = mid + 1
12         else:
13             right = mid - 1
14
15     return False
```

Coding: binary search

```python
def binary_search(items, target):
    left = 0
    right = len(items) - 1

    while left <= right:
        mid = (left + right) // 2

        if items[mid] == target:
            return True
        elif items[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return False
```

The code is short, but the assumption is strong: the list must be sorted by the same comparison used in the search.

Dynamic ordered search

Dynamic data makes repeated sorting expensive

Static data

Sort once, then binary search many times.

Dynamic data makes repeated sorting expensive

Static data

Sort once, then binary search many times.

Dynamic data

New records arrive, old records disappear, values are updated.

Dynamic data makes repeated sorting expensive

Static data

Sort once, then binary search many times.

Dynamic data

New records arrive, old records disappear, values are updated.

Operation on sorted array	Cost
search	$O(\log n)$
insert while keeping order	$O(n)$
delete while keeping order	$O(n)$
re-sort after update	usually wasteful

Dynamic data makes repeated sorting expensive

Static data

Sort once, then binary search many times.

Dynamic data

New records arrive, old records disappear, values are updated.

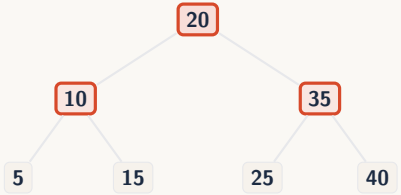
Operation on sorted array	Cost
search	$O(\log n)$
insert while keeping order	$O(n)$
delete while keeping order	$O(n)$
re-sort after update	usually wasteful

The issue is not only search. Dynamic systems also pay for keeping the structure searchable.

BSTs: keeping data searchable while it changes

Idea

A binary search tree stores ordered data without requiring array shifts.

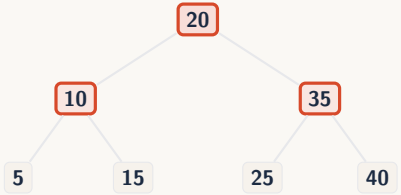


BSTs: keeping data searchable while it changes

Idea

A binary search tree stores ordered data without requiring array shifts.

- search follows comparisons;

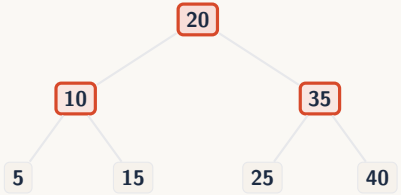


BSTs: keeping data searchable while it changes

Idea

A binary search tree stores ordered data without requiring array shifts.

- search follows comparisons;
- insertion follows comparisons;

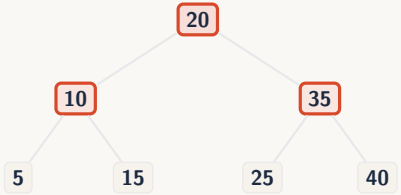


BSTs: keeping data searchable while it changes

Idea

A binary search tree stores ordered data without requiring array shifts.

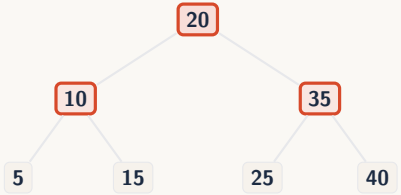
- search follows comparisons;
- insertion follows comparisons;
- balanced BST: $O(\log n)$ search, insert, delete.



BSTs: keeping data searchable while it changes

Idea

A binary search tree stores ordered data without requiring array shifts.



- search follows comparisons;
- insertion follows comparisons;
- balanced BST: $O(\log n)$ search, insert, delete.

BSTs fix repeated sorting partially, but the target remains $O(\log n)$ and balance must be maintained.

When $\log n$ may not cut it

n	n **checks**	$\log_2 n$ **checks**
1 000	1 000	≈ 10
1 000 000	1 000 000	≈ 20
1 000 000 000	1 000 000 000	≈ 30
10^{12}	10^{12}	≈ 40

When $\log n$ may not cut it

n	n **checks**	$\log_2 n$ **checks**
1 000	1 000	≈ 10
1 000 000	1 000 000	≈ 20
1 000 000 000	1 000 000 000	≈ 30
10^{12}	10^{12}	≈ 40

Where even small costs matter

Caches, routing tables, compilers, databases, authentication tokens, packet processing, real-time monitoring.

When $\log n$ may not cut it

n	n **checks**	$\log_2 n$ **checks**
1 000	1 000	≈ 10
1 000 000	1 000 000	≈ 20
1 000 000 000	1 000 000 000	≈ 30
10^{12}	10^{12}	≈ 40

Where even small costs matter

Caches, routing tables, compilers, databases, authentication tokens, packet processing, real-time monitoring.

$O(\log n)$ is fast. But millions of repeated exact lookups can make even logarithmic work worth avoiding.

Hashing

What we have so far

Binary search and BSTs improve search by using order.

From ordered search to key lookup

What we have so far

Binary search and BSTs improve search by using order.

Different kind of question

Many systems do not ask where an object belongs in sorted order. They ask:

"Does this exact key exist?"

From ordered search to key lookup

What we have so far

Binary search and BSTs improve search by using order.

Different kind of question

Many systems do not ask where an object belongs in sorted order. They ask:

"Does this exact key exist?"

- student ID $\rightarrow$ student record;

From ordered search to key lookup

What we have so far

Binary search and BSTs improve search by using order.

Different kind of question

Many systems do not ask where an object belongs in sorted order. They ask:

"Does this exact key exist?"

- student ID $\rightarrow$ student record;
- sensor ID $\rightarrow$ latest reading;

From ordered search to key lookup

What we have so far

Binary search and BSTs improve search by using order.

Different kind of question

Many systems do not ask where an object belongs in sorted order. They ask:

"Does this exact key exist?"

- student ID $\rightarrow$ student record;
- sensor ID $\rightarrow$ latest reading;
- variable name $\rightarrow$ symbol table entry;

From ordered search to key lookup

What we have so far

Binary search and BSTs improve search by using order.

Different kind of question

Many systems do not ask where an object belongs in sorted order. They ask:

"Does this exact key exist?"

- student ID $\rightarrow$ student record;
- sensor ID $\rightarrow$ latest reading;
- variable name $\rightarrow$ symbol table entry;
- session token $\rightarrow$ active session.

From ordered search to key lookup

What we have so far

Binary search and BSTs improve search by using order.

Different kind of question

Many systems do not ask where an object belongs in sorted order. They ask:

"Does this exact key exist?"

- student ID $\rightarrow$ student record;
- sensor ID $\rightarrow$ latest reading;
- variable name $\rightarrow$ symbol table entry;
- session token $\rightarrow$ active session.

This is search by identity, not search by order.

The dictionary ADT

Definition 5.1 Dictionary / Map

A dictionary stores key-value pairs and retrieves values using their keys.

```
grades = {}

grades["S1042"] = 17
grades["S2091"] = 14

print(grades["S1042"])
print("S2091" in grades)
```

The dictionary ADT

Definition 5.2 Dictionary / Map

A dictionary stores key-value pairs and retrieves values using their keys.

Main operations

- put(key, value)
- get(key)
- contains(key)
- delete(key)

```
grades = {}

grades["S1042"] = 17
grades["S2091"] = 14

print(grades["S1042"])
print("S2091" in grades)
```

The dictionary ADT

Definition 5.3 Dictionary / Map

A dictionary stores key-value pairs and retrieves values using their keys.

Main operations

- put(key, value)
- get(key)
- contains(key)
- delete(key)

```
grades = {}

grades["S1042"] = 17
grades["S2091"] = 14

print(grades["S1042"])
print("S2091" in grades)
```

The ADT describes what we want. The hash table is one possible implementation.

Naive dictionary implementation

Simplest idea

Store key-value pairs in a list.

$$[(\text{"}S1042\text{"}, 17),\ (\text{"}S2091\text{"}, 14),\ (\text{"}S7710\text{"}, 12)]$$

Naive dictionary implementation

Simplest idea

Store key-value pairs in a list.

$$[(\text{"}S1042\text{"}, 17),\ (\text{"}S2091\text{"}, 14),\ (\text{"}S7710\text{"}, 12)]$$

- get(key) scans until it finds the key;

Naive dictionary implementation

Simplest idea

Store key-value pairs in a list.

$$[(\text{"}S1042\text{"}, 17),\ (\text{"}S2091\text{"}, 14),\ (\text{"}S7710\text{"}, 12)]$$

- get(key) scans until it finds the key;
- contains(key) also scans;

Naive dictionary implementation

Simplest idea

Store key-value pairs in a list.

$$[(\text{"}S1042\text{"}, 17),\ (\text{"}S2091\text{"}, 14),\ (\text{"}S7710\text{"}, 12)]$$

- get(key) scans until it finds the key;
- contains(key) also scans;
- put(key, value) scans if we need to update an existing key;

Naive dictionary implementation

Simplest idea

Store key-value pairs in a list.

$$[(\text{"}S1042\text{"}, 17),\ (\text{"}S2091\text{"}, 14),\ (\text{"}S7710\text{"}, 12)]$$

- get(key) scans until it finds the key;
- contains(key) also scans;
- put(key, value) scans if we need to update an existing key;
- delete(key) scans to find the key first.

Naive dictionary implementation

Simplest idea

Store key-value pairs in a list.

$$[(\text{"}S1042\text{"}, 17),\ (\text{"}S2091\text{"}, 14),\ (\text{"}S7710\text{"}, 12)]$$

- get(key) scans until it finds the key;
- contains(key) also scans;
- put(key, value) scans if we need to update an existing key;
- delete(key) scans to find the key first.

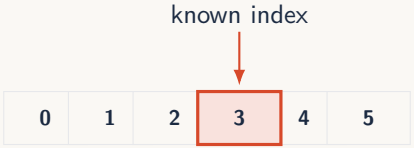
The naive dictionary is useful, but its main operations are $\Theta(n)$.

Arrays give direct access

Key observation

If we already know the index, array access is constant time.

$$a[i] \quad \Rightarrow \quad \Theta(1)$$

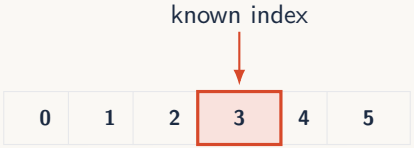


Arrays give direct access

Key observation

If we already know the index, array access is constant time.

$$a[i] \quad \Rightarrow \quad \Theta(1)$$



The problem: our keys are not array indices. We need a way to convert a key into an index.

Hash functions

Hash functions

Definition 6.1 Hash function

A hash function converts a key into an integer.

$$h(\text{key}) \rightarrow \text{integer}$$

Hash functions

Definition 6.2 Hash function

A hash function converts a key into an integer.

$$h(\text{key}) \rightarrow \text{integer}$$

- deterministic: same key, same result;

Hash functions

Definition 6.3 Hash function

A hash function converts a key into an integer.

$$h(\text{key}) \rightarrow \text{integer}$$

- deterministic: same key, same result;
- fast to compute;

Hash functions

Definition 6.4 Hash function

A hash function converts a key into an integer.

$$h(\text{key}) \rightarrow \text{integer}$$

- deterministic: same key, same result;
- fast to compute;
- uses the whole key;

Hash functions

Definition 6.5 Hash function

A hash function converts a key into an integer.

$$h(\text{key}) \rightarrow \text{integer}$$

- deterministic: same key, same result;
- fast to compute;
- uses the whole key;
- spreads keys well.

Bad hash function

Problem

This function sends every key to the same place.

```
1 def bad_hash(key, m):
2     return 0
```

All keys get the same value.

Weak hash function

Problem

This function uses all characters, but ignores their order.

```
1 def weak_hash(text, m):
2     total = 0
3     for char in text:
4         total += ord(char)
5     return total % m
```

For example, `"AB"` and `"BA"` produce the same sum.

Better simple hash function

Improvement

This version uses both the characters and their order.

```
def poly_hash(text, m):
    h = 0
    base = 257

    for char in text:
        h = (h * base + ord(char)) % m

    return h
```

Python's built-in `hash`

Built-in function

`hash(x)` returns an integer hash value for object `x`.

```
1 print(hash(42))
2 print(hash("S1042"))
3 print(hash((1, "A")))
```

Python's built-in `hash`

Built-in function

`hash(x)` returns an integer hash value for object `x`.

```
1 print(hash(42))
2 print(hash("S1042"))
3 print(hash((1, "A")))
```

- used internally by dictionaries and sets;

Python's built-in `hash`

Built-in function

`hash(x)` returns an integer hash value for object `x`.

```
1 print(hash(42))
2 print(hash("S1042"))
3 print(hash((1, "A")))
```

- used internally by dictionaries and sets;
- only works for hashable objects;

Python's built-in `hash`

Built-in function

`hash(x)` returns an integer hash value for object `x`.

```
1 print(hash(42))
2 print(hash("S1042"))
3 print(hash((1, "A")))
```

- used internally by dictionaries and sets;
- only works for hashable objects;
- equal objects must have equal hashes;

Python's built-in `hash`

Built-in function

`hash(x)` returns an integer hash value for object `x`.

```
1 print(hash(42))
2 print(hash("S1042"))
3 print(hash((1, "A")))
```

- used internally by dictionaries and sets;
- only works for hashable objects;
- equal objects must have equal hashes;
- different objects may still have the same hash.

Python's built-in `hash`

Built-in function

`hash(x)` returns an integer hash value for object `x`.

```
print(hash(42))
print(hash("S1042"))
print(hash((1, "A")))
```

- used internally by dictionaries and sets;
- only works for hashable objects;
- equal objects must have equal hashes;
- different objects may still have the same hash.

Python hashing is type-specific: integers, strings, tuples, and custom objects do not all hash the same way.

The magic method `__hash__`

Main idea

When Python evaluates:

```
1 hash(x)
```

The magic method `__hash__`

Main idea

When Python evaluates:

```
1 hash(x)
```

it calls the object's hash behavior:

```
1 x.__hash__()
```

The magic method `__hash__`

Main idea

When Python evaluates:

```
1 hash(x)
```

it calls the object's hash behavior:

```
1 x.__hash__()
```

- `__hash__` must return an integer;

The magic method `__hash__`

Main idea

When Python evaluates:

```
1 hash(x)
```

it calls the object's hash behavior:

```
1 x.__hash__()
```

- `__hash__` must return an integer;
- dictionaries use it to locate keys;

The magic method `__hash__`

Main idea

When Python evaluates:

```
1 hash(x)
```

it calls the object's hash behavior:

```
1 x.__hash__()
```

- `__hash__` must return an integer;
- dictionaries use it to locate keys;
- sets use it to locate elements;

The magic method `__hash__`

Main idea

When Python evaluates:

```
1 hash(x)
```

it calls the object's hash behavior:

```
1 x.__hash__()
```

- `__hash__` must return an integer;
- dictionaries use it to locate keys;
- sets use it to locate elements;
- if `__hash__` is missing or disabled, the object is unhashable.

The equality-hash contract

Required rule

If two objects are equal, they must have the same hash.

$$a == b \implies hash(a) == hash(b)$$

The equality-hash contract

Required rule

If two objects are equal, they must have the same hash.

$$a == b \implies hash(a) == hash(b)$$

Not required

The reverse is not guaranteed.

$$hash(a) == hash(b) \not\implies a == b$$

Custom objects: default behavior

Default object behavior

If a class does not redefine equality, objects compare by identity.

```
class Student:
    pass

a = Student()
b = Student()

print(a == b)        # False
print(hash(a))
print(hash(b))
```

Custom objects: default behavior

Default object behavior

If a class does not redefine equality, objects compare by identity.

```
class Student:
    pass

a = Student()
b = Student()

print(a == b)        # False
print(hash(a))
print(hash(b))
```

- by default, different instances are different keys;

Custom objects: default behavior

Default object behavior

If a class does not redefine equality, objects compare by identity.

```
class Student:
    pass

a = Student()
b = Student()

print(a == b)        # False
print(hash(a))
print(hash(b))
```

- by default, different instances are different keys;
- the inherited hash is consistent with identity equality;

Custom objects: default behavior

Default object behavior

If a class does not redefine equality, objects compare by identity.

```
class Student:
    pass

a = Student()
b = Student()

print(a == b)        # False
print(hash(a))
print(hash(b))
```

- by default, different instances are different keys;
- the inherited hash is consistent with identity equality;
- this is valid, but often not the behavior we want.

Custom objects: define equality and hash

Example

Two students should be considered equal if they have the same student ID.

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name

    def __eq__(self, other):
        return (
            isinstance(other, Student)
            and self.student_id == other.student_id
        )

    def __hash__(self):
        return hash(self.student_id)
```

Custom objects: define equality and hash

Example

Two students should be considered equal if they have the same student ID.

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name

    def __eq__(self, other):
        return (
            isinstance(other, Student)
            and self.student_id == other.student_id
        )

    def __hash__(self):
        return hash(self.student_id)
```

Use the same information for equality and hashing. Here, both depend on `student_id`.

Common mistake: `__eq__` without `__hash__`

Problem

If a class defines `__eq__` but not `__hash__`, Python makes its instances unhashable.

```
1 class Student:
2     def __eq__(self, other):
3         return True
4
5 s = Student()
6
7 print(hash(s))       # TypeError
```

Common mistake: `__eq__` without `__hash__`

Problem

If a class defines `__eq__` but not `__hash__`, Python makes its instances unhashable.

```
1 class Student:
2     def __eq__(self, other):
3         return True
4
5 s = Student()
6
7 print(hash(s))       # TypeError
```

- equality was redefined;

Common mistake: `__eq__` without `__hash__`

Problem

If a class defines `__eq__` but not `__hash__`, Python makes its instances unhashable.

```
1 class Student:
2     def __eq__(self, other):
3         return True
4
5 s = Student()
6
7 print(hash(s))       # TypeError
```

- equality was redefined;
- Python no longer knows which hash rule is safe;

Common mistake: `__eq__` without `__hash__`

Problem

If a class defines `__eq__` but not `__hash__`, Python makes its instances unhashable.

```
class Student:
    def __eq__(self, other):
        return True

s = Student()

print(hash(s))      # TypeError
```

- equality was redefined;
- Python no longer knows which hash rule is safe;
- the object cannot be used as a dictionary key or set element.

Common mistake: `__eq__` without `__hash__`

Problem

If a class defines `__eq__` but not `__hash__`, Python makes its instances unhashable.

```
1 class Student:
2     def __eq__(self, other):
3         return True
4
5 s = Student()
6
7 print(hash(s))       # TypeError
```

- equality was redefined;
- Python no longer knows which hash rule is safe;
- the object cannot be used as a dictionary key or set element.

Redefining equality forces you to think about hashing.

Hash mutable objects carefully

Danger

A dictionary key must keep the same hash while it is inside the dictionary.

```
class Student:
    def __init__(self, student_id):
        self.student_id = student_id

    def __eq__(self, other):
        return self.student_id == other.student_id

    def __hash__(self):
        return hash(self.student_id)
```

```
s = Student("S1042")
d = {s: "registered"}

s.student_id = "S9999"     # dangerous
```

String hashes are randomized

Important detail

Python salts hashes of strings and bytes with a random value.

```
1 print(hash("S1042"))
```

String hashes are randomized

Important detail

Python salts hashes of strings and bytes with a random value.

```
1 print(hash("S1042"))
```

- stable during one Python process;

String hashes are randomized

Important detail

Python salts hashes of strings and bytes with a random value.

```
1 print(hash("S1042"))
```

- stable during one Python process;
- not guaranteed to be the same across different runs;

String hashes are randomized

Important detail

Python salts hashes of strings and bytes with a random value.

```
1 print(hash("S1042"))
```

- stable during one Python process;
- not guaranteed to be the same across different runs;
- helps defend against adversarial collision attacks;

String hashes are randomized

Important detail

Python salts hashes of strings and bytes with a random value.

```
1 print(hash("S1042"))
```

- stable during one Python process;
- not guaranteed to be the same across different runs;
- helps defend against adversarial collision attacks;
- can be controlled with `PYTHONHASHSEED` for repeatable runs.

String hashes are randomized

Important detail

Python salts hashes of strings and bytes with a random value.

```
1 print(hash("S1042"))
```

- stable during one Python process;
- not guaranteed to be the same across different runs;
- helps defend against adversarial collision attacks;
- can be controlled with `PYTHONHASHSEED` for repeatable runs.

Do not store Python's built-in hash values as permanent IDs.

Hash tables

Hash table ADT

Definition 7.1 Hash table

A hash table stores key-value pairs and supports fast exact lookup by key.

Hash table ADT

Definition 7.2 Hash table

A hash table stores key-value pairs and supports fast exact lookup by key.

Core operations

- `put(key, value);`
- `get(key);`
- `contains(key);`
- `delete(key).`

Hash table ADT

Definition 7.3 Hash table

A hash table stores key-value pairs and supports fast exact lookup by key.

Core operations

- `put(key, value);`
- `get(key);`
- `contains(key);`
- `delete(key).`

Expected performance

With a good hash function and controlled load factor:

$$\text{search, insert, delete} = O(1) \text{ average}$$

Hash table ADT

Definition 7.4 Hash table

A hash table stores key-value pairs and supports fast exact lookup by key.

Core operations

- `put(key, value);`
- `get(key);`
- `contains(key);`
- `delete(key).`

Expected performance

With a good hash function and controlled load factor:

$$\text{search, insert, delete} = O(1) \text{ average}$$

The ADT looks simple. What about the underlying data structure?

Hash table idea

Definition 7.5 Hash table

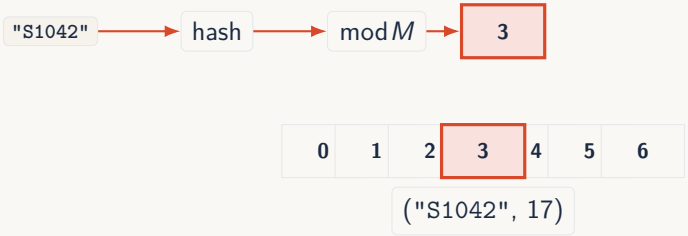
A hash table implements a dictionary using an array and a hash function.

("S1042", 17)

Hash table idea

Definition 7.6 Hash table

A hash table implements a dictionary using an array and a hash function.



Lookup repeats the same computation, goes to the same position, then verifies the key.

From hash value to table index

Table size

If the table has M slots, we convert the hash value into a valid index:

$$\text{index} = h(\text{key}) \bmod M$$

From hash value to table index

Table size

If the table has M slots, we convert the hash value into a valid index:

$$\text{index} = h(\text{key}) \bmod M$$

The modulo step forces the result to be one of the table positions.

Table size, memory, and load factor

Notation

N = number of stored keys $\qquad$ M = number of table slots

Table size, memory, and load factor

Notation

N = number of stored keys $\qquad$ M = number of table slots

Load factor

$$\alpha = \frac{N}{M}$$

Table size, memory, and load factor

Notation

N = number of stored keys $\qquad$ M = number of table slots

Load factor

$$\alpha = \frac{N}{M}$$

- larger M usually means fewer collisions;

Table size, memory, and load factor

Notation

N = number of stored keys $\qquad$ M = number of table slots

Load factor

$$\alpha = \frac{N}{M}$$

- larger M usually means fewer collisions;
- smaller α usually means shorter searches;

Table size, memory, and load factor

Notation

N = number of stored keys $\qquad$ M = number of table slots

Load factor

$$\alpha = \frac{N}{M}$$

- larger M usually means fewer collisions;
- smaller α usually means shorter searches;
- but larger M uses more memory;

Table size, memory, and load factor

Notation

N = number of stored keys M = number of table slots

Load factor

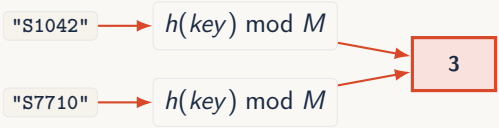
$$\alpha = \frac{N}{M}$$

- larger M usually means fewer collisions;
- smaller α usually means shorter searches;
- but larger M uses more memory;
- hash tables trade memory for speed.

Collisions are inevitable

Definition 7.7 Collision

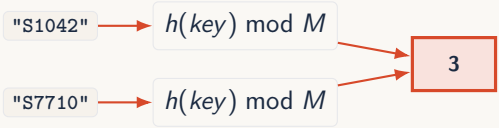
A collision happens when two different keys map to the same table index.



Collisions are inevitable

Definition 7.8 Collision

A collision happens when two different keys map to the same table index.

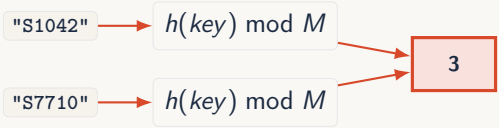


- if $N > M$, collision is guaranteed;

Collisions are inevitable

Definition 7.9 Collision

A collision happens when two different keys map to the same table index.

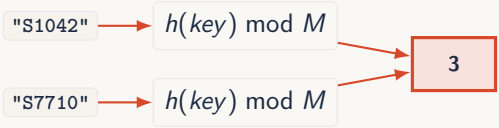


- if $N > M$, collision is guaranteed;
- this is the pigeonhole principle;

Collisions are inevitable

Definition 7.10 Collision

A collision happens when two different keys map to the same table index.

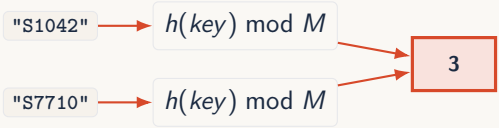


- if $N > M$, collision is guaranteed;
- this is the pigeonhole principle;
- even if $N \leq M$, collision is still possible;

Collisions are inevitable

Definition 7.11 Collision

A collision happens when two different keys map to the same table index.



- if $N > M$, collision is guaranteed;
- this is the pigeonhole principle;
- even if $N \leq M$, collision is still possible;
- a hash table must define a collision strategy.

Collision resolution: chaining

Collision resolution by chaining

Definition 8.1 Chaining

Each table slot stores a bucket, usually a list of key-value pairs.

Idea

If several keys map to the same index, store all of them in the same bucket.

0	[]
1	[("S1042",17),("S7710",12)]
2	[]
3	[("S2091",14)]

A collision does not overwrite the old key. It adds another pair to the bucket.

Chaining invariant

Invariant

A key-value pair with key `k` belongs in bucket:

$$i = h(k) \bmod M$$

- every key has one target bucket;
- each bucket may contain zero, one, or many pairs;
- inside a bucket, we still compare actual keys;
- the hash chooses the bucket;
- equality confirms the exact key.

Hashing narrows the search to one bucket. It does not remove the need for key comparison.

Operation: `put(key, value)`

Goal

Insert a new key-value pair or update the value if the key already exists.

1. compute $\texttt{index} = h(\texttt{key}) \bmod M$;
2. go to that bucket;
3. scan the bucket for the key;
4. if the key exists, replace its value;
5. otherwise, append a new key-value pair.

Chaining handles collision by allowing several pairs to live in the same bucket.

Chaining: put

```
def put(table, key, value):
    index = hash(key) % len(table)
    bucket = table[index]

    for i in range(len(bucket)):
        existing_key, existing_value = bucket[i]

        if existing_key == key:
            bucket[i] = (key, value)
            return

    bucket.append((key, value))
```

The bucket may contain collisions, so insertion must check whether the key is already present.

Operation: `get(key)`

Goal

Return the value associated with a key.

1. compute $\texttt{index} = h(\texttt{key}) \bmod M$;
2. go to that bucket;
3. scan the key-value pairs in the bucket;
4. compare each stored key with the searched key;
5. return the matching value, or report that the key is absent.

The hash gives the bucket. Equality finds the exact key inside that bucket.

Chaining: `get`

```
def get(table, key):
    index = hash(key) % len(table)
    bucket = table[index]

    for existing_key, value in bucket:
        if existing_key == key:
            return value

    raise KeyError(key)
```

If the bucket has length k, this lookup costs $\Theta(k)$.

Operation: `contains(key)`

Goal

Answer whether the key exists in the table.

1. compute the target bucket;
2. scan only that bucket;
3. return `True` if a matching key is found;
4. return `False` otherwise.

`contains` is the same search as `get`, but it returns a Boolean instead of a value.

Chaining: `contains`

```
def contains(table, key):
    index = hash(key) % len(table)
    bucket = table[index]

    for existing_key, value in bucket:
        if existing_key == key:
            return True

    return False
```

Operation: `delete(key)`

Goal

Remove the key-value pair associated with a key.

1. compute the target bucket;
2. scan the bucket for the key;
3. if found, remove that pair from the bucket;
4. if absent, report failure or do nothing, depending on the ADT design.

Deletion is simple in chaining because removing one pair does not disturb other buckets.

Chaining: `delete`

```
def delete(table, key):
    index = hash(key) % len(table)
    bucket = table[index]

    for i in range(len(bucket)):
        existing_key, value = bucket[i]

        if existing_key == key:
            bucket.pop(i)
            return

    raise KeyError(key)
```

Chaining complexity

Let

$$N = \text{number of keys}, \qquad M = \text{number of buckets}, \qquad \alpha = \frac{N}{M}$$

Operation	Average with good hashing	Worst case
`put`	$\Theta(1+\alpha)$	$\Theta(n)$
`get`	$\Theta(1+\alpha)$	$\Theta(n)$
`contains`	$\Theta(1+\alpha)$	$\Theta(n)$
`delete`	$\Theta(1+\alpha)$	$\Theta(n)$

If the load factor stays controlled, chaining gives expected constant-time operations.

Collision resolution: open addressing

Collision resolution by open addressing

Definition 9.1 Open addressing

All key-value pairs are stored directly inside the table. If the target slot is occupied, we search for another slot.

Idea

A collision does not create a bucket. It starts a probe sequence.

Open addressing keeps the table as one array, but makes insertion, lookup, and deletion more delicate.

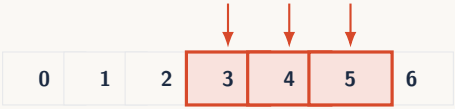
Linear probing

Probe sequence

If the first index is i, linear probing tries:

$$i,\ i+1,\ i+2,\ i+3, \ldots$$

wrapping around the table when needed.



Linear probing is simple, but it can create clusters of occupied slots.

Open addressing slot states

Each slot has a state

A slot is not just full or empty. With deletion, we need three states.

State	Meaning
`EMPTY`	no key has ever occupied this slot
`OCCUPIED`	the slot currently stores a key-value pair
`TOMBSTONE`	a key was deleted here, but probing must continue

Tombstones are the key to safe deletion in open addressing.

Operation: `get(key)`

Goal

Return the value associated with a key.

1. compute the first index $i = h(\texttt{key}) \bmod M$;
2. inspect slots along the probe sequence;
3. if the key is found, return its value;
4. if a tombstone is found, continue probing;
5. if an empty slot is found, stop: the key is absent.

An empty slot stops search. A tombstone does not.

Open addressing: get

```
EMPTY = None
TOMBSTONE = object()

def get(table, key):
    start = hash(key) % len(table)
    index = start

    while table[index] is not EMPTY:
        if table[index] is not TOMBSTONE:
            existing_key, value = table[index]

            if existing_key == key:
                return value

        index = (index + 1) % len(table)

        if index == start:
            break

    raise KeyError(key)
```

Operation: `contains(key)`

Goal

Answer whether the key exists in the table.

1. follow the same probe sequence as `get`;
2. return `True` if the key is found;
3. skip tombstones;
4. stop only at an empty slot or after a full loop;
5. return `False` if no match is found.

`contains` is `get` without returning the value.

Open addressing: `contains`

```
def contains(table, key):
    start = hash(key) % len(table)
    index = start

    while table[index] is not EMPTY:
        if table[index] is not TOMBSTONE:
            existing_key, value = table[index]

            if existing_key == key:
                return True

        index = (index + 1) % len(table)

        if index == start:
            break

    return False
```

Operation: `put(key, value)`

Goal

Insert a new key-value pair or update the value if the key already exists.

1. compute the first index;
2. probe until the key, an empty slot, or a reusable tombstone is found;
3. if the key exists, update its value;
4. if a tombstone appears, remember its position;
5. insert into the first remembered tombstone, otherwise into the first empty slot.

Insertion may reuse tombstones, but it must still continue probing to avoid duplicating an existing key.

Open addressing: put – probing

```
def put(table, key, value):
    start = hash(key) % len(table)
    index = start
    first_tombstone = None

    while table[index] is not EMPTY:
        if table[index] is TOMBSTONE:
            if first_tombstone is None:
                first_tombstone = index
        else:
            existing_key, old_value = table[index]

            if existing_key == key:
                table[index] = (key, value)
                return

        index = (index + 1) % len(table)

        if index == start:
            break
```

Open addressing: `put` – insertion

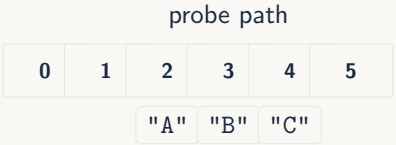
```
    if first_tombstone is not None:
        table[first_tombstone] = (key, value)

    elif table[index] is EMPTY:
        table[index] = (key, value)

    else:
        raise RuntimeError("table is full")
```

Reuse the first tombstone found. If there is no tombstone, use the first empty slot.

Why deletion is tricky

Wrong idea

Do not simply replace a deleted key by `EMPTY`.



If `"B"` is replaced by `EMPTY`, searching for `"C"` may stop too early.

Tombstones preserve the probe path

Correct idea

When deleting, mark the slot as `TOMBSTONE` instead of `EMPTY`.

0	1	2	3	4	5
`"A"`	`TOMB`	`"C"`			

- search continues through tombstones;
- insertion may reuse tombstones;
- empty slots still mean the key was never further in this probe chain.

Operation: `delete(key)`

Goal

Remove the key-value pair without breaking future searches.

1. compute the first index;
2. follow the probe sequence;
3. if the key is found, replace the slot by `TOMBSTONE`;
4. if a tombstone is found, continue probing;
5. if an empty slot is found, stop: the key is absent.

Deletion in open addressing removes the value, but keeps a marker so the probe chain remains valid.

Open addressing: `delete`

```
def delete(table, key):
    start = hash(key) % len(table)
    index = start

    while table[index] is not EMPTY:
        if table[index] is not TOMBSTONE:
            existing_key, value = table[index]

            if existing_key == key:
                table[index] = TOMBSTONE
                return

        index = (index + 1) % len(table)

        if index == start:
            break

    raise KeyError(key)
```

Tombstones: useful but not free

Problem

Too many tombstones make probe sequences longer.

- tombstones preserve correctness after deletion;
- but searches must step through them;
- insertions can reuse them;
- if many deleted slots remain, performance degrades;
- practical hash tables periodically resize or rebuild the table.

Tombstones fix correctness. Resizing or rebuilding fixes accumulated clutter.

Open addressing complexity

Load factor

$$\alpha = \frac{N}{M}$$

Open addressing needs $\alpha < 1$, because all keys live inside the table.

Operation	Average, low load	Worst case
`put`	expected $\Theta(1)$	$\Theta(n)$
`get`	expected $\Theta(1)$	$\Theta(n)$
`contains`	expected $\Theta(1)$	$\Theta(n)$
`delete`	expected $\Theta(1)$	$\Theta(n)$

Open addressing is fast only while the table is not crowded and tombstones are controlled.

Conclusion

Search strategies: the big picture

Structure	Search	Insert	Best use case
Unsorted list	$\Theta(n)$	$\Theta(1)$	small or unstructured data
Sorted array	$\Theta(\log n)$	$\Theta(n)$	static ordered data
Balanced BST	$\Theta(\log n)$	$\Theta(\log n)$	dynamic ordered data
Hash table	expected $\Theta(1)$	expected $\Theta(1)$	exact lookup by key

The right structure depends on what kind of search the system needs.

Order search vs identity search

Order search

Use when comparisons matter.

- minimum / maximum;
- next larger value;
- range queries;
- sorted traversal.

Identity search

Use when exact keys matter.

- student ID;
- sensor ID;
- session token;
- variable name.

Hashing is for identity lookup. Sorting and trees are for order.

Hash tables: what must work together

- a dictionary ADT defines the operations;
- an array gives direct access by index;
- a hash function converts keys into indexes;
- the table size controls memory and load factor;
- a collision strategy preserves correctness.

A hash table is not just a hash function. It is a full design: table, hash function, collision handling, and resizing policy.

Chaining vs open addressing

Question	Chaining	Open addressing
Where are collisions stored?	bucket lists	inside the array
Deletion	simple	needs tombstones
Load factor	can exceed 1	must stay below 1
Search path	one bucket	probe sequence
Main risk	long buckets	clustering and tombstones

Both methods are correct. Their performance depends on good hashing and controlled crowding.

What to remember

- linear search makes no assumptions, but costs $\Theta(n)$;
- binary search is fast, but requires sorted data;
- BSTs support dynamic ordered search;
- hash tables target exact lookup by key;
- collisions are unavoidable;
- chaining stores collisions in buckets;
- open addressing resolves collisions by probing;
- tombstones are necessary for safe deletion in open addressing.